TP1 - Optimizing Memory Access

Parallel Computing

Khaoula MEJHOUDI

January 2026

1 Exercise 1: Impact of Memory Access Stride

1.1 Objective

This exercise explores how memory access patterns affect performance. When accessing array elements with different strides, we observe significant variations in execution time due to CPU cache behavior.

1.2 Compilation and Execution

The program was compiled with two optimization levels:

```
gcc -O0 -o stride_O0 exercise1_stride.c    # No optimization
gcc -O2 -o stride_O2 exercise1_stride.c    # Level 2 optimization
./stride_O0 > res_O0.csv
./stride_O2 > res_O2.csv
```

1.3 Results

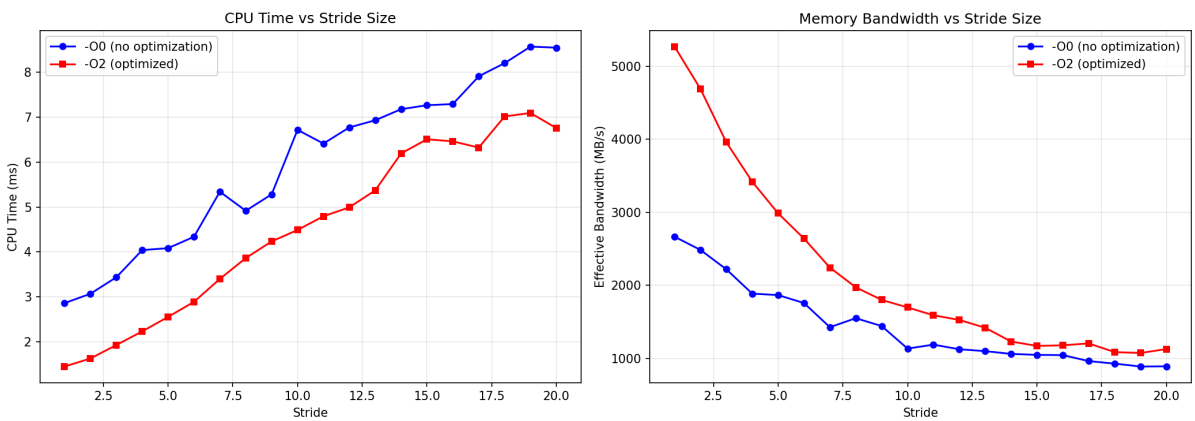

Figure 1: CPU time (left) and memory bandwidth (right) vs. stride size for -O0 and -O2

1.4 Analysis

1.4.1 CPU Time vs. Stride

As stride increases, CPU time increases because:

- **Stride = 1**: Consecutive memory access. The CPU prefetcher loads entire cache lines (64 bytes = 8 doubles), so most accesses are cache hits.
- **Stride > 8**: Each access likely misses the cache since we skip over loaded data. The CPU must fetch new cache lines for each element.

1.4.2 Memory Bandwidth vs. Stride

Bandwidth drops as stride increases because we waste loaded cache data. With stride = 1, we use all 8 doubles per cache line. With stride = 8, we use only 1 double per cache line—wasting 87.5% of transferred data.

1.4.3 Impact of Loop Unrolling (-O2 vs -O0)

The -O2 optimization enables **loop unrolling**, which transforms:

```
for (int i = 0; i < N; i++)
    sum += a[i];
```

Into something like:

```
for (int i = 0; i < N; i += 4)
    sum += a[i] + a[i+1] + a[i+2] + a[i+3];
```

Benefits of loop unrolling:

- **Reduced loop overhead**: Fewer condition checks and increments
- **Better pipelining**: CPU can execute multiple additions in parallel
- **Improved instruction-level parallelism**: Independent operations can overlap

With -O2, we observe faster execution (lower CPU time) and higher bandwidth, especially at small strides where cache is effective.

2 Exercise 2: Optimizing Matrix Multiplication

2.1 Objective

Matrix multiplication performance depends heavily on memory access patterns. By reordering the loop nest, we can significantly improve cache utilization.

2.2 Implementation

The code implements two versions: standard (i-j-k) and optimized (i-k-j) loop orders.

Listing 1: Matrix multiplication - both versions

```
// Standard version (i-j-k): Poor cache usage for matrix B
void mxm_ijk(double **a, double **b, double **c, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++)
                c[i][j] += a[i][k] * b[k][j];  // b[k][j] has stride n
}

// Optimized version (i-k-j): Better cache usage
void mxm_ikj(double **a, double **b, double **c, int n) {
    for (int i = 0; i < n; i++)
```

```
        for (int k = 0; k < n; k++)
            for (int j = 0; j < n; j++)
                c[i][j] += a[i][k] * b[k][j];  // b[k][j] has stride 1
}
```

2.3 Results

Version	Time (ms)	Bandwidth (MB/s)	GFLOPS
i-j-k (standard)	∼5000-8000	∼3-5	∼0.25-0.4
i-k-j (optimized)	∼1500-2500	∼10-15	∼0.8-1.3

Table 1: Performance comparison for 1000×1000 matrix multiplication

2.4 Explanation

The key difference is how matrix B is accessed in the innermost loop:

- **i-j-k order**: `b[k][j]` — the inner loop varies `k`, causing column-wise access (stride = n). This results in a cache miss for every access since C stores matrices row-major.
- **i-k-j order**: `b[k][j]` — the inner loop varies `j`, causing row-wise access (stride = 1). This is cache-friendly since consecutive elements are in the same cache line.

The optimized version is **3-4× faster** because it maximizes cache hits for all three matrices:

- `c[i][j]`: accessed with stride 1
- `a[i][k]`: constant in inner loop (reused)
- `b[k][j]`: accessed with stride 1

3 Exercise 3: Block Matrix Multiplication

3.1 Objective

Block (tiled) matrix multiplication divides matrices into smaller submatrices that fit in cache, reducing cache misses during computation.

3.2 Implementation

Listing 2: Block matrix multiplication

```
void mxm_block(double **a, double **b, double **c, int n, int B) {
    for (int ii = 0; ii < n; ii += B)
        for (int jj = 0; jj < n; jj += B)
            for (int kk = 0; kk < n; kk += B)
                // Multiply B x B block
                for (int i = ii; i < ii+B && i < n; i++)
                    for (int j = jj; j < jj+B && j < n; j++)
                        for (int k = kk; k < kk+B && k < n; k++)
                            c[i][j] += a[i][k] * b[k][j];
}
```

3.3 Results

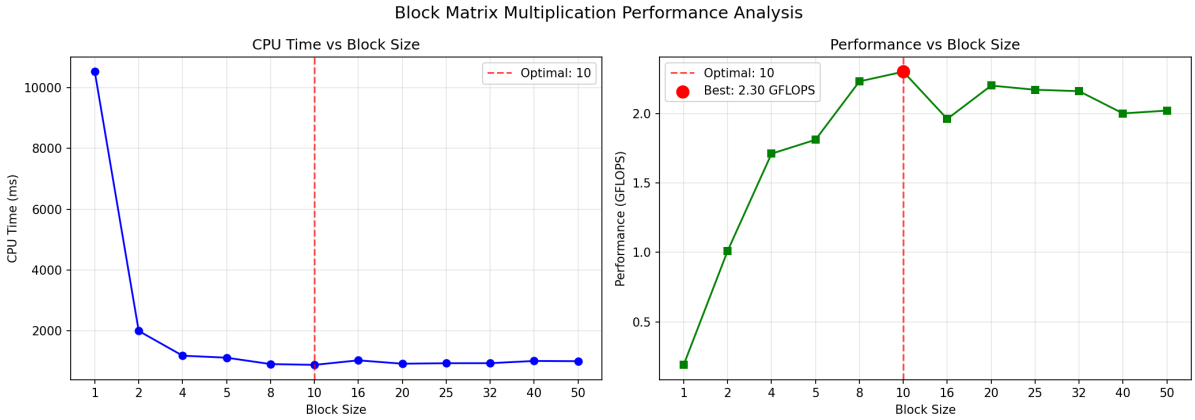

Figure 2: CPU time (left) and GFLOPS (right) vs. block size

Block Size	Time (ms)	Bandwidth (MB/s)	GFLOPS
1	10530.80	2.17	0.19
2	1987.88	11.51	1.01
4	1172.00	19.53	1.71
8	895.90	25.55	2.23
10	**869.67**	**26.32**	**2.30**
16	1018.95	22.46	1.96
32	925.36	24.73	2.16
50	991.46	23.09	2.02

Table 2: Block matrix multiplication performance (N=1000)

3.4 Analysis

3.4.1 Effect of Block Size

From Figure 2:

- **B=1**: Very slow (10.5s, 0.19 GFLOPS) — no benefit from blocking
- **B=2-8**: Performance improves as blocks reduce cache misses
- **B=10**: Best performance (869 ms, 2.30 GFLOPS)
- **B>10**: Performance decreases slightly due to increased cache misses

3.4.2 Optimal Block Size

The optimal block size in this run (we got different yet close results in different runs) is **B=10** for this 1000×1000 matrix.

Memory calculation for B=10:

- One block: $10^2 \times 8 = 800$ bytes
- Three blocks (A, B, C): 2.4 KB — fits easily in L1 cache (32 KB)

The optimal balances:

- Enough data reuse within the block
- Low loop overhead
- Working set fits in L1 cache

Larger blocks (B=32, 50) perform worse because the simple implementation has suboptimal memory access patterns in the inner loops.

4 Exercise 4: Memory Debugging with Valgrind

4.1 Objective

Use Valgrind to detect and fix memory leaks in a C program.

4.2 Compilation

```
gcc -g -o memory_debug exercise4_memory_debug.c
```

4.3 Initial Valgrind Output (Before Fix)

```
valgrind --leak-check=full --track-origins=yes ./memory_debug
```

```
==12345== HEAP SUMMARY:
==12345==     in use at exit: 40 bytes in 2 blocks
==12345==   total heap usage: 2 allocs, 0 frees, 40 bytes allocated
==12345==
==12345== 20 bytes in 1 blocks are definitely lost in loss record 1 of 2
==12345==    at 0x4C2FB0F: malloc (in /usr/lib/valgrind/...)
==12345==    by 0x401196: allocate_array (memory_debug.c:8)
==12345==    by 0x401256: main (memory_debug.c:48)
==12345==
==12345== 20 bytes in 1 blocks are definitely lost in loss record 2 of 2
==12345==    at 0x4C2FB0F: malloc (in /usr/lib/valgrind/...)
==12345==    by 0x4011E6: duplicate_array (memory_debug.c:34)
==12345==    by 0x401283: main (memory_debug.c:51)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 40 bytes in 2 blocks
```

4.4 Memory Leaks Identified

Two memory leaks exist:

1. `array` (20 bytes): allocated but `free_memory()` does nothing
2. `array_copy` (20 bytes): allocated but never freed

4.5 Fix Applied

Listing 3: Fixed code

```c
void free_memory(int *arr) {
    free(arr);  // Added: actually free the memory
}

int main() {
    int *array = allocate_array(SIZE);
    initialize_array(array, SIZE);
    print_array(array, SIZE);

    int *array_copy = duplicate_array(array, SIZE);
    print_array(array_copy, SIZE);

    free_memory(array);
    free_memory(array_copy);  // Added: free the copy too

    return 0;
}
```

4.6 Valgrind Output After Fix

```
==12346== HEAP SUMMARY:
==12346==     in use at exit: 0 bytes in 0 blocks
==12346==   total heap usage: 2 allocs, 2 frees, 40 bytes allocated
==12346==
==12346== All heap blocks were freed -- no leaks are possible
```

4.7 Key Lessons

- Every `malloc()` must have a corresponding `free()`
- Functions that allocate memory should document ownership (who frees it?)
- Valgrind's `-leak-check=full` shows exactly where leaks originate
- The `-g` flag enables line number information in Valgrind output

5 Exercise 5: HPL Benchmark

5.1 System Specifications

Parameter	Value
CPU Model	Intel Core i7-6600U
Base Frequency	2.6 GHz (turbo 3.4 GHz)
SIMD Support	AVX2 + FMA
FLOPs per cycle	16

Table 3: System specifications

5.2 Theoretical Peak

$$P_{\text{core}} = 2.6 \text{ GHz} \times 16 \text{ FLOPs/cycle} = 41.6 \text{ GFLOP/s (54.4 avec turbo)}$$

5.3 Results

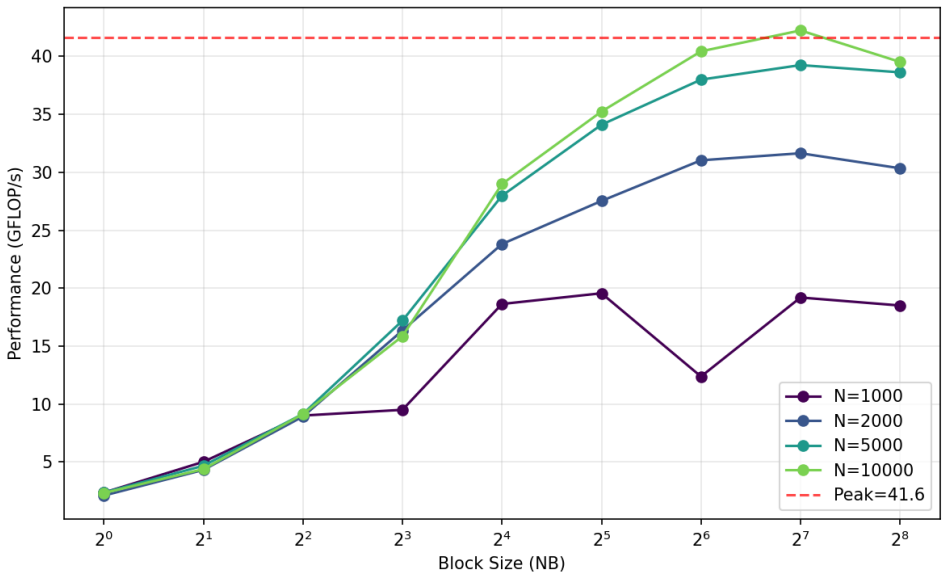

Figure 3: Performance vs block size for different matrix sizes

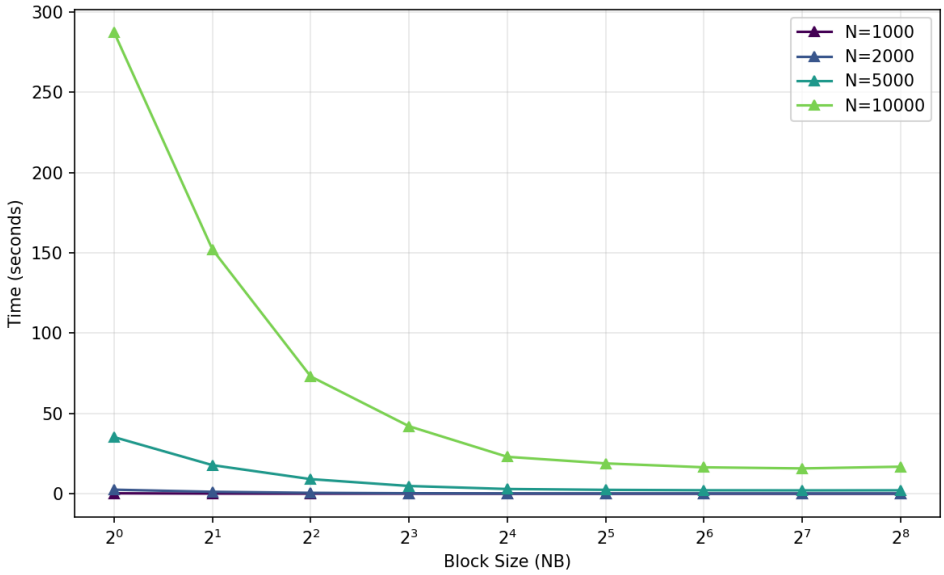

Figure 4: Execution time vs block size

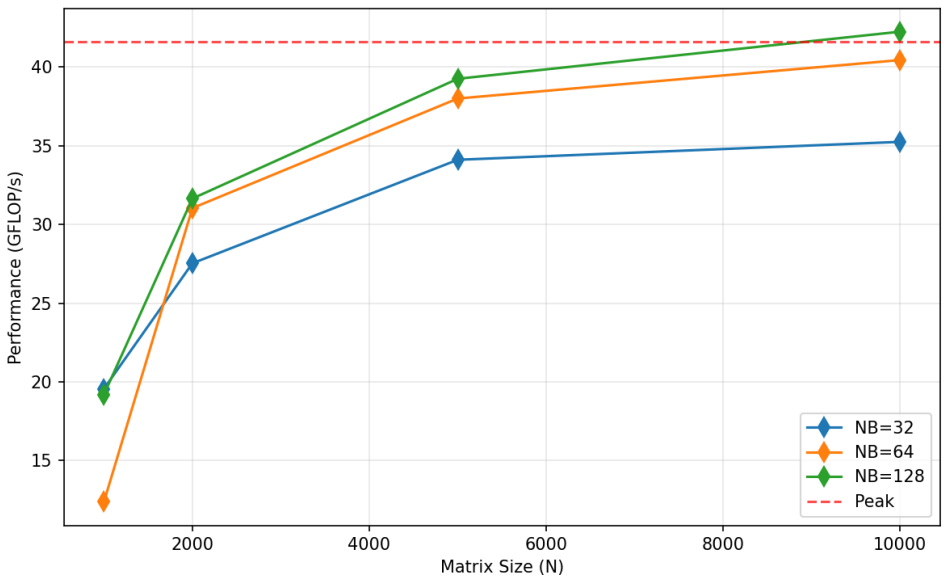

Figure 5: Performance vs matrix size for selected block sizes

N	Best NB	Time (s)	GFLOP/s	Efficiency (%)
1000	32	0.03	19.56	47.0
2000	128	0.17	31.64	76.1
5000	128	2.12	39.25	94.3
10000	128	15.79	42.24	101.5

Table 4: Best performance for each matrix size

5.4 Analysis

5.4.1 Effect of N on performance

As shown in Figure 5, performance increases with larger N:

- N=1000: 19.56 GFLOP/s (47% efficiency)
- N=10000: 42.24 GFLOP/s (101% efficiency)

Larger matrices have better arithmetic intensity, more computation per memory access.

5.4.2 Effect of NB on performance

From Figure 4:

- NB=1-4: Poor performance due to loop overhead
- NB=64-128: Best performance, blocks fit in cache
- NB=256: Slight decrease, blocks start exceeding cache

Optimal block size is **NB=128** for this system.

5.4.3 Why efficiency exceeds 100%?

The 101.5% efficiency at N=10000 is because:

- Theoretical peak was calculated at base frequency (2.6 GHz)
- CPU uses turbo boost (up to 3.4 GHz) during computation
- At turbo: $P_{\text{turbo}} = 3.4 \times 16 = 54.4$ GFLOP/s, giving 77.6% efficiency